

2025-1

Basic Algorithm Study For ■ Beginners

기초 알고리즘 스터디 3주차

01

시간복잡도

- Big-O notation
- 시간복잡도 계산하기
- 공간복잡도

02

브루트포스

- 정의
- 어떤 때에 사용할까?

03

백트래킹

- 정의
- 재귀함수로 구현하기

시간복잡도

—

01



01

Big-O notation

Big-O notation

성장 속도의 상한?

Big-O notation

$$O(g(n)) = f(n)$$

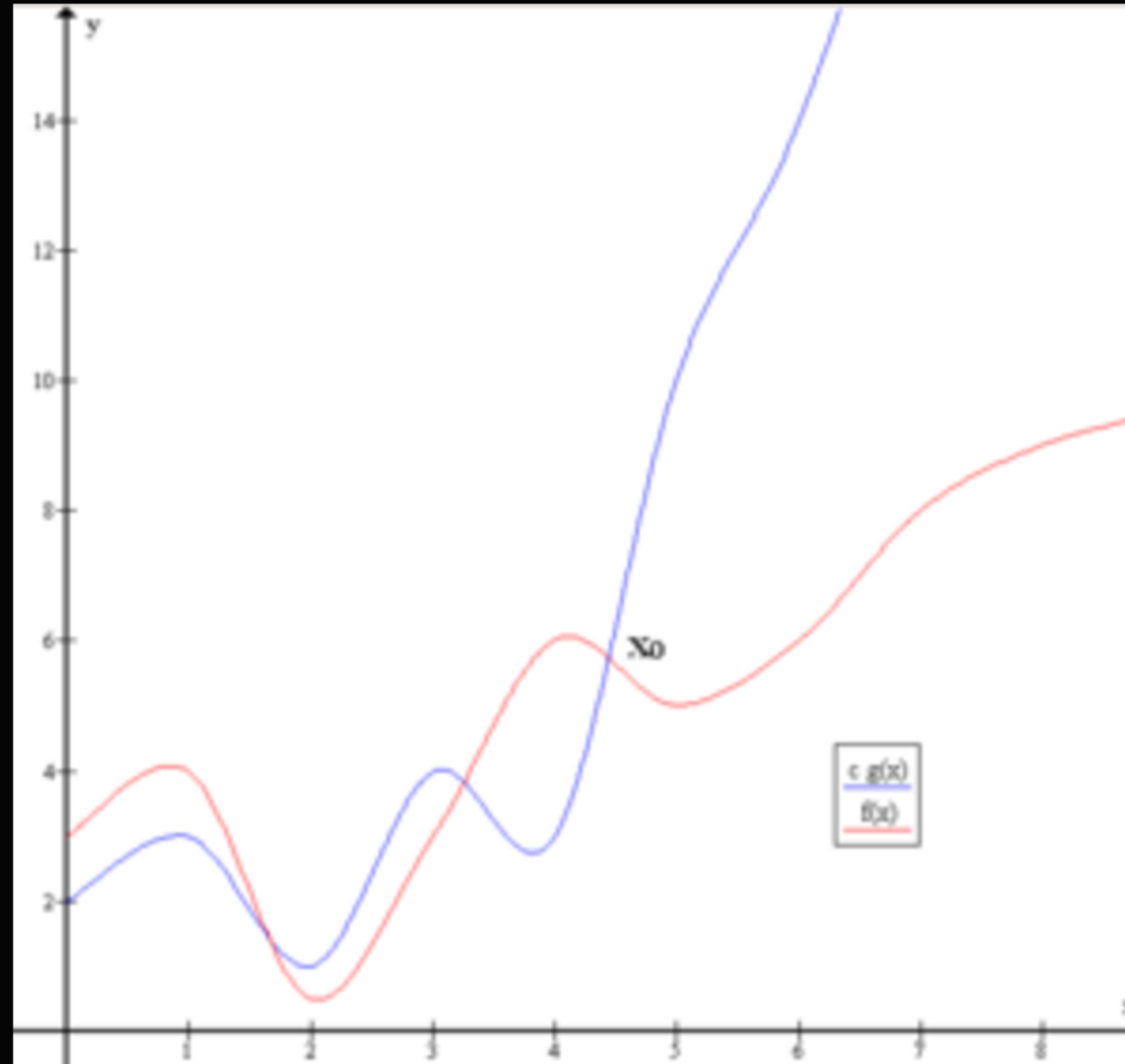
어떤 함수 $g(n)$ 과 $f(n)$ 이

$x \geq k$ 인 모든 x 에 대해 $0 \leq f(x) \leq c * g(x)$ 인 c 와 k 가 존재함.

최악의 경우에도 $c * g(n)$ 보다 빠르다.

01

Big-O notation



$c \cdot g(x)$ (파란 선)이 $k=4$ 를 기점으로 $f(x)$ 보다 항상 큼.
즉 $O(g(x)) = f(x)$

01

Big-O notation

그렇다면

$$x^2 + x = O(2x^2)$$

$$x^2 + x = O(x^3) \text{ 인가??}$$

$$x^2 + x, 2x^2, x^3$$



01

Big-O notation

$$x^2 + x = O(2x^2)$$

$$x^2 + x = O(x^3) \text{ 인가???}$$

그렇다! Big-O는 상한이지
최소상한이 아니다.

$$x^2 + x, 2x^2, x^3$$



하지만 보통은 Big-O를 얘기할 때
 $x^2 + x = O(x^2)$ 라고 하지 $O(x^3)$ 이라고 하지는 않는다.

사실 더 정확한 정의로 Big- Θ (빅-세타, 상한이자 하한)가 있지만
다들 적당히 Big-O라고 한다.
아무래도 구분하기 귀찮은 듯 하다.

가장 간단한 방법이 있다.

Big-O를 말할 때는

시간복잡도 식에서 가장 빠르게 증가하는 항을 기준으로 말하자.

$N!$, $k^N \succ N^k \succ \sqrt{N} \succ \log(N) \succ k$



01

시간복잡도 계산하기

CPU는 1초에 대략 1억번의 계산을 수행한다.
이를 바탕으로 수행 시간을 예측하면서 문제를 풀어보자!



01

시간복잡도 계산하기

```
N = int(input().strip())  
res = 0  
for _ in range(N):  
    res += 1
```



01

시간복잡도 계산하기

```
N = int(input().strip())  
res = 0  
for _ in range(N):  
    res += 1
```

=O(N), 선형 시간

01

시간복잡도 계산하기

```
N = int(input().strip())  
res = 0  
for _ in range(N):  
    for _ in range(N):  
        res += 1
```

= $O(N^2)$, 2차 시간

01

시간복잡도 계산하기

```
N = int(input().strip())  
res = 0  
while N > 0:  
    N //= 2  
    res += 1
```

=O(log N), 로그 시간



01

공간복잡도

```
N = int(input().strip())
```

```
lst = [0]*N
```

=O(N), 선형 공간

공간복잡도는 시간복잡도에 비해 널널한 경우가 많다.
대회에 따라서는 1024MB로 통일하는 경우도 있다.
왜냐하면 메모리가 시간보다 훨씬 싸기 때문이다!
그러니 특수한 경우가 아니라면 신경쓰지 말도록 하자.

참고사항:

일반적으로 pypy가 python보다 빠르다.

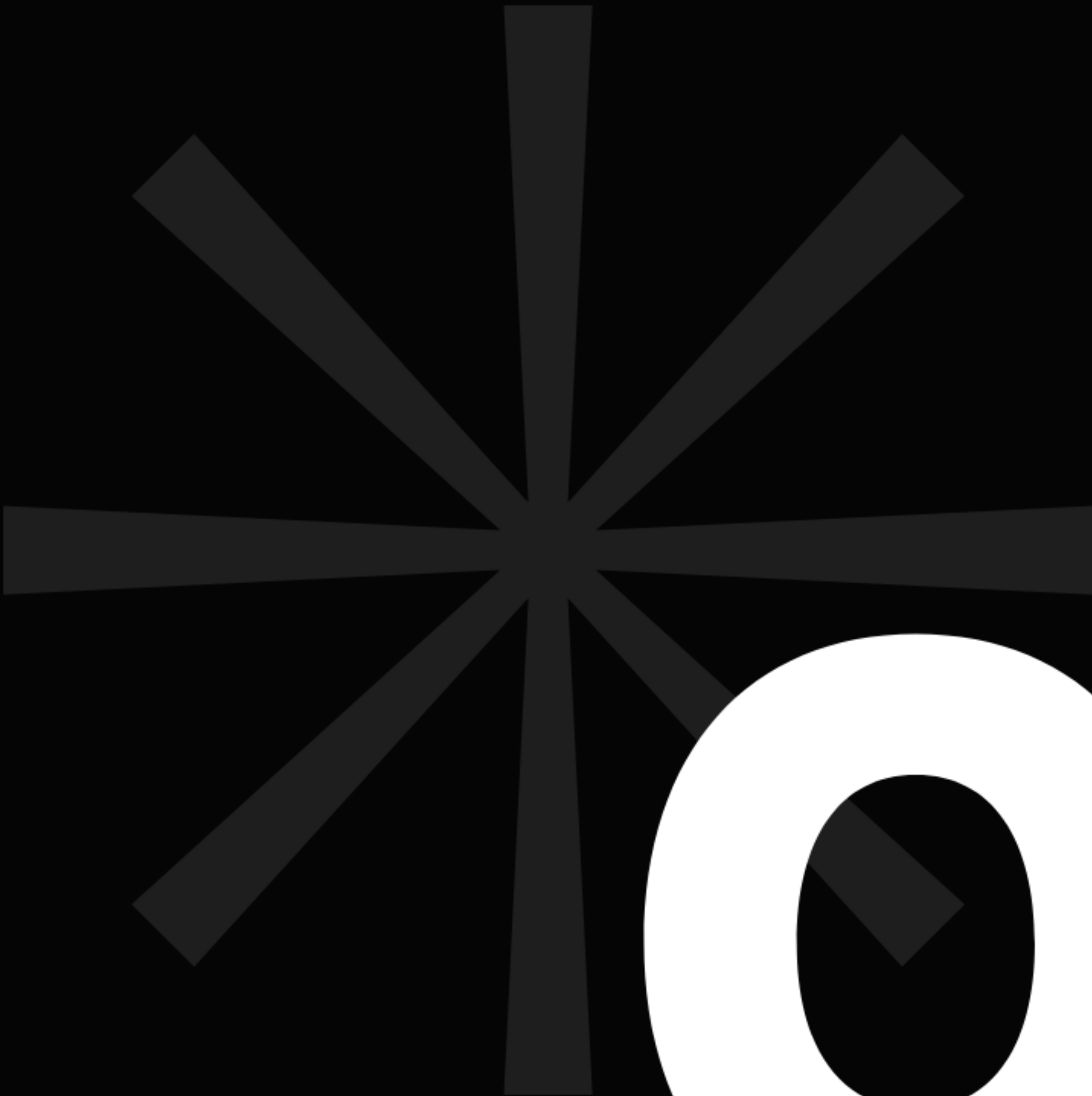
간단한 코드는 python이 더 빠를 수도 있지만,

코드가 길어질수록 메모리를 더 희생하는 대신에 속도를 얻는다.

어지간한 경우에는 pypy를 사용하도록 하자.

버킷
포스

—

 02



02

정의

브루트포스

Brute Force

짐승같은 힘



02

정의

노가다

장점

- 머리를 덜 써도 됨.
- 100% 정답을 찾아낼 수 있음.
- 구현이 어렵지 않음.



02

정의

단점

- 시간이 오래 걸림.

0부터 9까지의 숫자로 이루어진
N자리의 금고 비밀번호를 노가다로 맞춰 봅시다!



02

정의

$N = 1$, 10회 반복

$N = 2$, 100회 반복

$N = 3$, 1000회 반복...

$N = K$, 10^K 회 반복

$= O(10^N)$



02

어떤 때에 사용할까?



브루트포스에서 가장 중요한 것은?

상황 파악

=시간복잡도 계산



02

어떤 때에 사용할까?

연습문제: #19532 수학은 비대면강의입니다

x,y의 범위는 대충 2000 정도이다.

이를 토대로 계산해보면

$2,000 * 2,000 = 4,000,000$ 이므로

충분히 통과 가능해 보인다!

02

어떤 때에 사용할까?

```
def solve():  
    a, b, c, d, e, f = map(int, input().split())  
    for i in range(-999, 1000):  
        for j in range(-999, 1000):  
            if a*i + b*j == c and d*i + e*j == f:  
                print(i, j)  
                return
```

solve()

32412 KB

404 ms

32412 KB

884 ms

깨알 팁: 같은 코드라도 함수 안에서 실행할 때 더 빠르다.



02

어떤 때에 사용할까?

연습문제: #2798 블랙잭

최악의 경우에 가능한 모든 경우의 수는

$$100 C 3 = 161,700$$

시간제한은 1초

이 정도면 널널하게 통과할 수 있을 것 같다!

그렇다면...

중복되지 않도록

3장의 카드를 뽑는 경우의 수를 만드는 방법만 알면
문제를 풀 수 있다.

어떤 때에 사용할까?

```
for i in range(0, N-2):  
    for j in range(i+1, N-1):  
        for k in range(j+1, N):  
            s = lst[i] + lst[j] + lst[k]  
            if M >= s > res:  
                res = s
```

$i < j < k$ 기 때문에
중복되지 않고 세 카드의 조합을 만들 수 있다.

하지만 이런 방식으로는
뽑는 카드의 수가 많아지면 대처하기 곤란해진다.

itertools 에 있는 combinations를 사용해보자.

combinations(이터러블 객체, 뽑는 크기)

combinations([1, 2, 3], 2) # (1, 2), (2, 3), (1, 3)

어떤 때에 사용할까?

```
import sys
input = sys.stdin.readline
from itertools import combinations

N, M = map(int, input().split())
lst = list(map(int, input().split()))
res = -1
for i, j, k in combinations(range(N), 3):
    s = lst[i] + lst[j] + lst[k]
    if M >= s > res:
        res = s

print(res)
```


combinations([1, 2, 3], 2) # (1, 2), (2, 3), (1, 3)
여기서 반환되는 객체의 속성은 이터레이터이다.

이터러블 객체 속의 원소가 요리라고 생각했을 때
이터레이터는 각 인덱스에 맞는 레시피만 갖고 있는
상태라고 생각할 수 있다.

이터레이터를 `list()`를 이용해 리스트로 바꿔주려면
레시피에 맞는 요리를 각각 만들어서 리스트에 담는 작업이 필요하다.

반대로 `for`문을 이용해 바로 반복을 돌리면 리스트에 담지 않고 바로바로 요리를 만들어서 사용한 뒤 버릴 수 있다!

02

어떤 때에 사용할까?

44060 KB	104 ms	Python 3 / 수정
32412 KB	80 ms	Python 3 / 수정

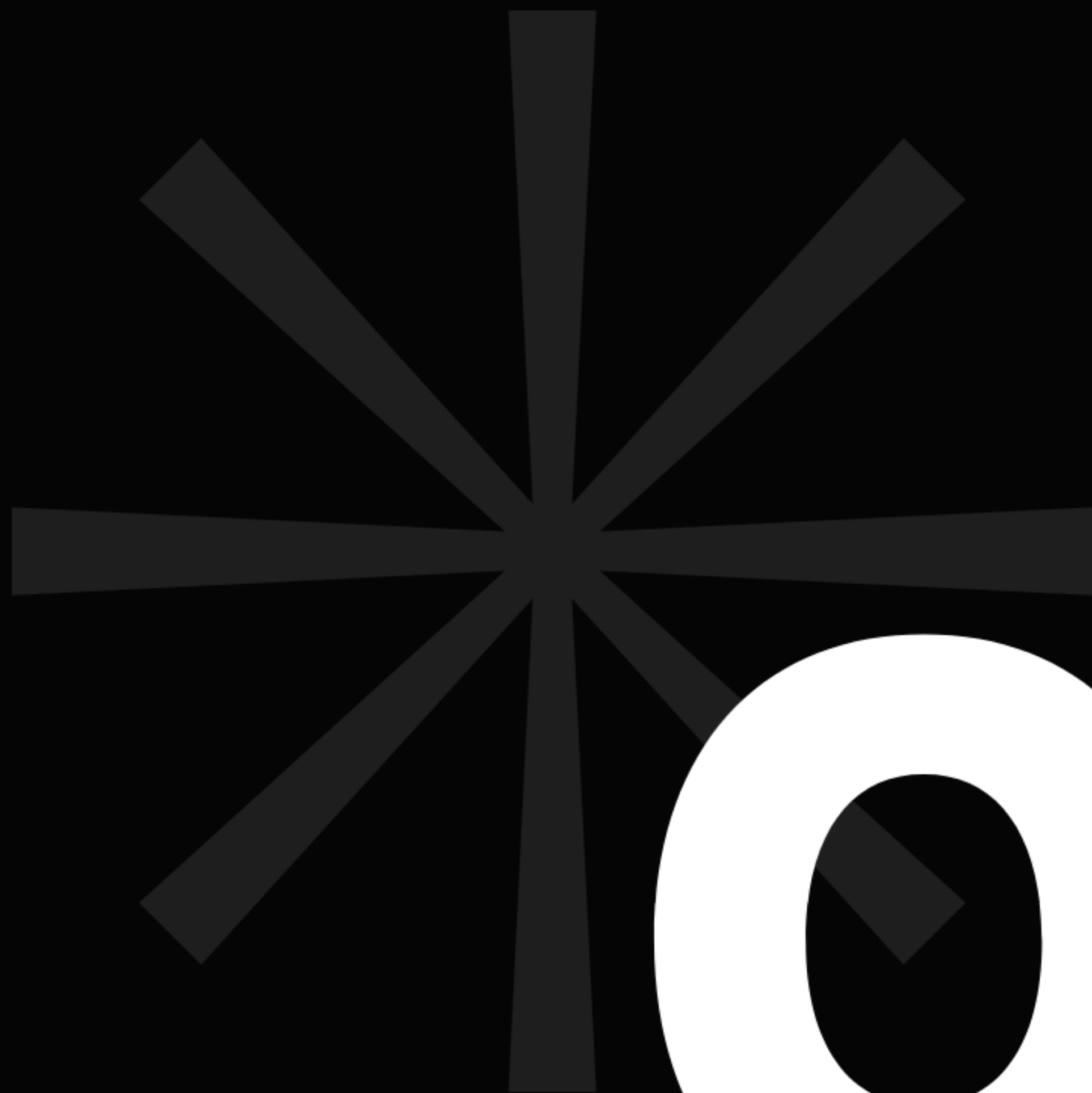
이터레이터를 리스트로 바꾼 뒤 순회할 때

이터레이터 상태 그대로 순회할 때

바로 사용하는 것이 메모리와 속도 면에서 효율적이다.

[]를 이용해 원소에 임의접근할 필요가 없다면 굳이 리스트로 바꾸지 말고 써버리자.

백트래킹



03

03

정의





03

정의



꼬치의 달인

<http://codeforces.com/group/2w09icwSjD/contest/588641/problem/D>

[« Back to main form](#)

꼬치의 달인 마스터

time limit per test: 5 seconds

memory limit per test: 1024 megabytes

꼬치의 달인은 **2002**년에 독일에서 만들어진 보드게임으로 플레이 방법은 아주 간단하다. 주문 카드를 보고 최대한 빠르게 조합에 맞는 꼬치를 만들기만 하면 된다.

이 게임을 하던 가은이는 문득 주어진 재료를 전부 사용해서 만들 수 있는 꼬치의 종류는 몇 개일지 궁금해졌다. 재료를 꽃은 순서가 다르면 다른 꼬치이지만, 가은이는 맛의 밸런스를 중시하기 때문에 같은 재료를 연속해서 꽃은 꼬치는 꼬치로 치지 않는다.

그러나 꼬치의 달인은 스피드가 생명이기 때문에 가은이는 이 문제에 대해 깊이 고민할 시간이 없었다. 여러분이 문제를 대신 해결해주자.

Input

첫째 줄에 피망, 새우, 토마토, 스테이크 블록의 수가 공백으로 구분되어 주어진다. 입력되는 모든 수는 **0** 이상 **4** 이하의 정수이다.

Output

첫째 줄에 만들 수 있는 꼬치의 가짓수를 출력한다.

Examples

input	
1 0 0 0	
output	
1	

최대로 주어지는 재료의 양은 $4*4 = 16$ 개

16개를 줄세우는 경우의 수는 $16P16 = 16! = 20,922,789,888,000$

대충 계산해도 209,227 초가 걸린다...

시간제한은 5초이다.

최대로 주어지는 재료의 양은 $4*4 = 16$ 개

16개를 줄세우는 경우의 수는 $16P16 = 16! = 20,922,789,888,000$

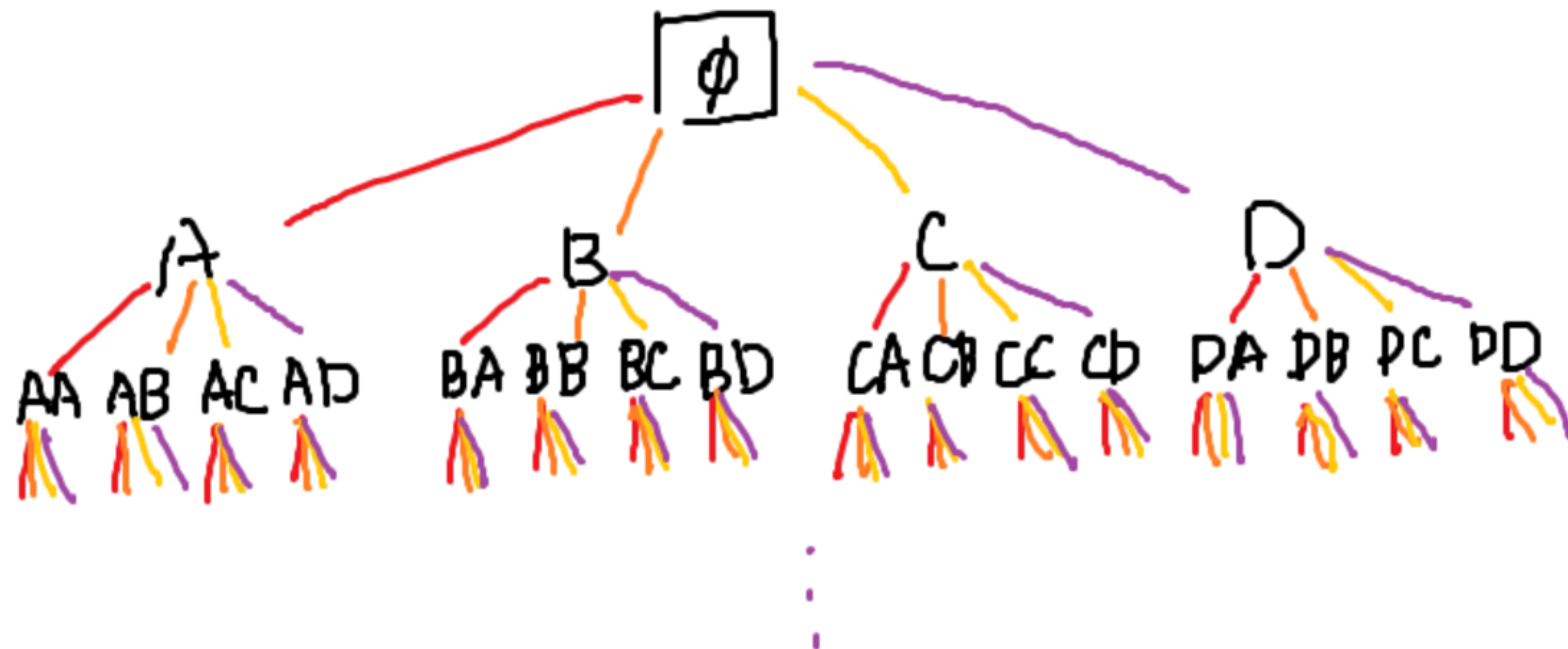
같은 재료끼리는 구분하지 않으므로 $(4!)^4$ 로 나눈다 하더라도
조건을 만족하는 꼬치를 판별하기 위해 꼬치당 16번의 연산이 필요하므로
아무래도 불가능해 보인다.

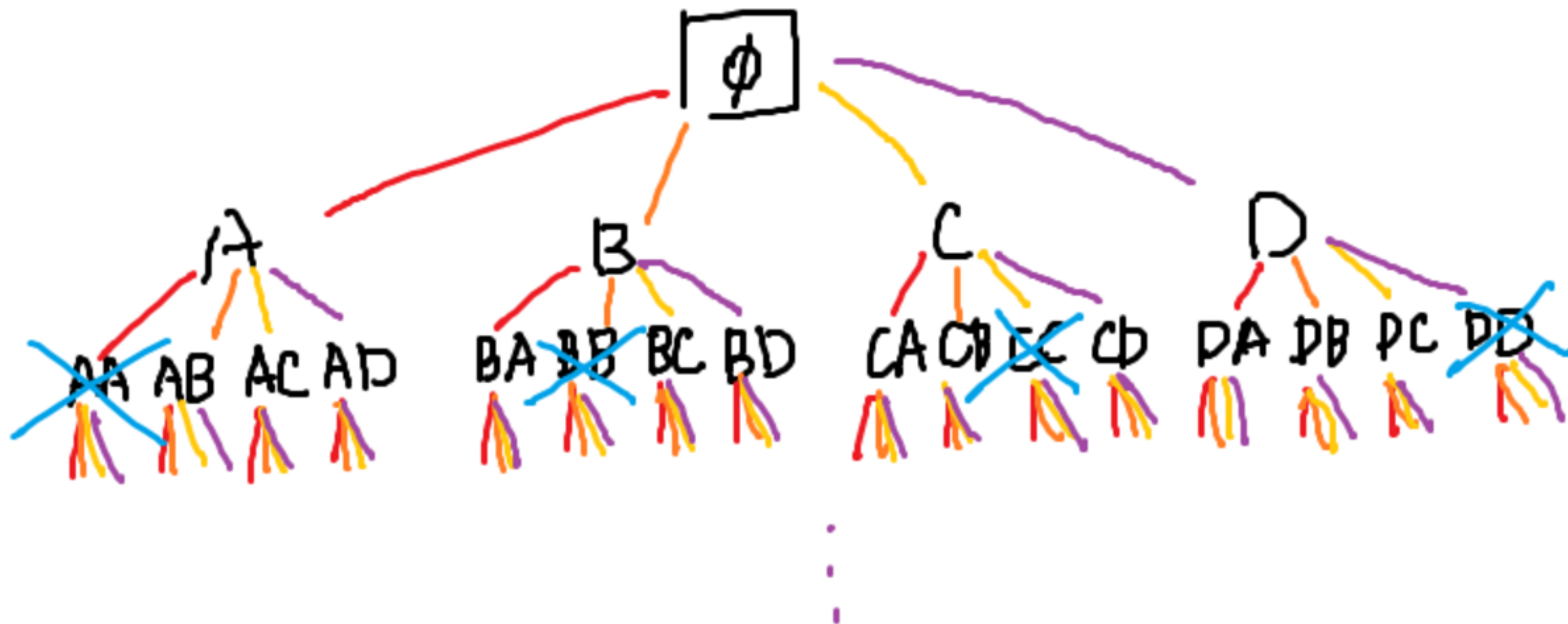
만약 4개의 재료 중 한개를 선택해서 16번 꺾은 뒤 검사한다면?

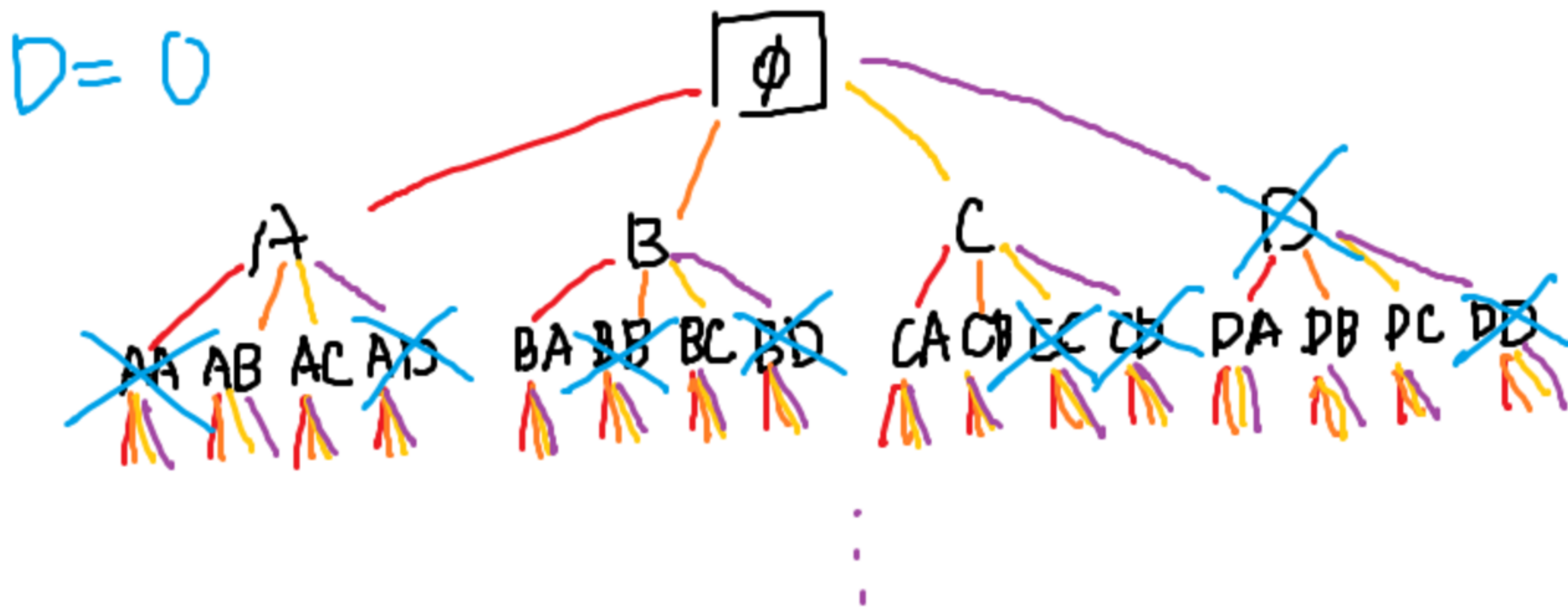
$$4^{16} = 4,294,967,296$$

이래도 곤란한 것은 마찬가지다.

하지만 실제로 '꼬치' 라고 불리는 경우의 수는 많지 않을 것 같다.
그렇다면 $16!$ 개의 경우를 모두 돌아볼 필요 없이,
'꼬치'를 만족하는 경우의 수만 세는 방법이 있을까?



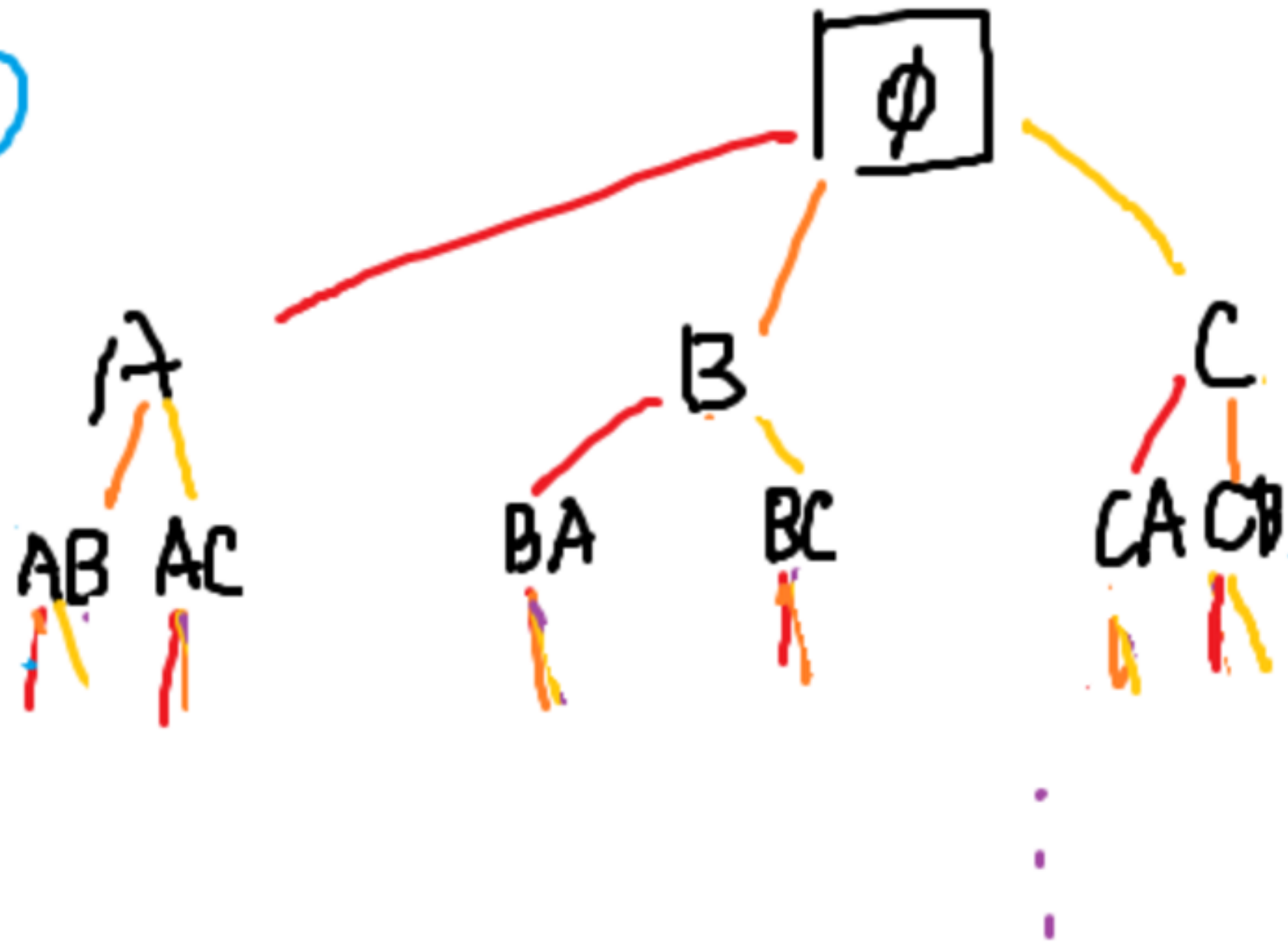




03

정의

$D = 0$





03

정의

BackTrack

안 되면 돌아가기

특징:

재귀함수로 구현

더 진행할 수 없으면 이전 단계로 되돌아감

가지치기를 이용해 탐색 횟수를 줄임

조건을 만족하면 return

장점

- 머리를 덜 써도 됨.
- 100% 정답을 찾아낼 수 있음.
- 이전에 했던 계산을 반복하지 않아도 됨.

시간복잡도 == 노드의 수

한 단계에 최대 3개의 가지로 분할

재료의 수가 N 일 때 재귀 깊이는 N 을 넘을 수 없으므로
최악의 경우 $O(3^N)$ 정도가 됨을 알 수 있다.

$N \leq 16$ 이므로 $3^{16} = 43,046,721$

여기서 가지치기를 진행하면 더 줄어든 것이다.

백트래킹 호출을 그림으로 그려보면
아까와 같은 나무 모양이 되는 것을 알 수 있다.
특정 상태에서 정해진 여러가지 다음 상태가 있을 때 쓰기 적절하다!



03

구현하기

수도코드(Sudo Code)를 작성해보자.

```
def backtrack(현재상태):
```

```
    if 정답에 도달했다면:
```

```
        정답 처리
```

```
        return
```

```
    for 가능한 다음 선택지:
```

```
        상태 변경
```

```
        backtrack(다음상태)
```

```
        상태 변경 취소
```

if 정답에 도달했다면:

정답 처리

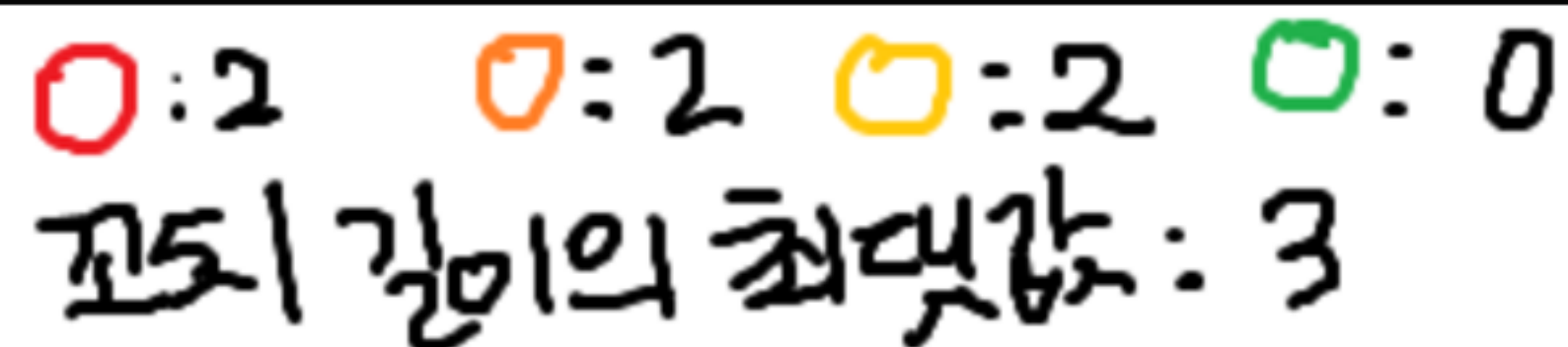
return

for 가능한 다음 선택지:

상태 변경

backtrack(다음상태)

상태 변경 취소





03

구현하기

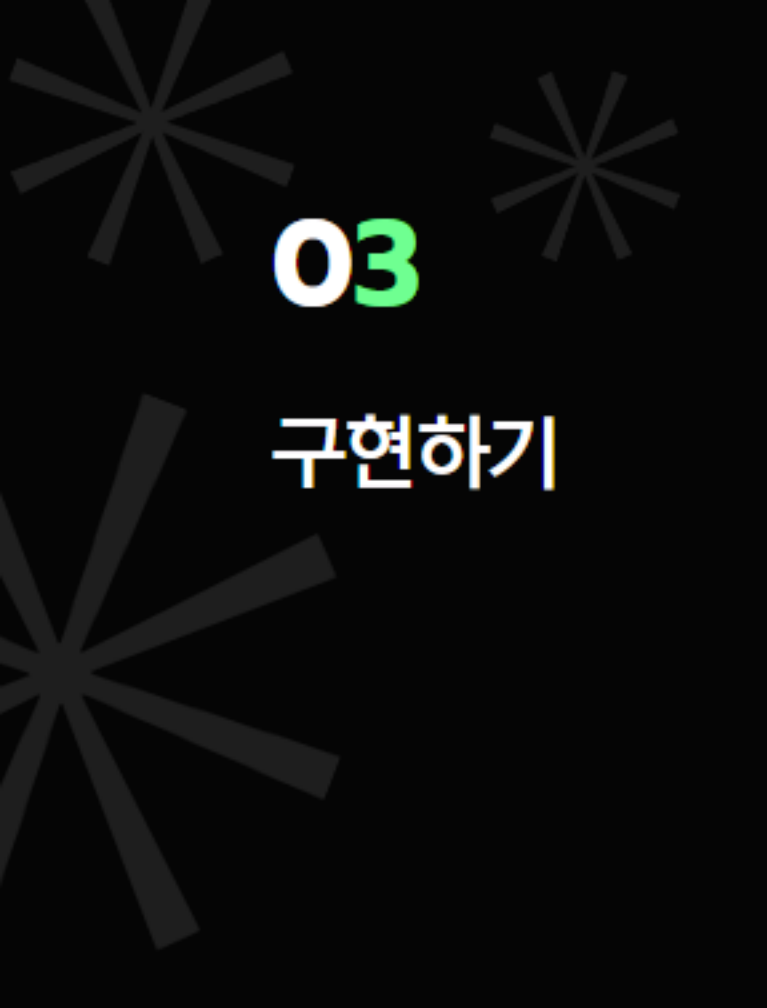


함수를 호출하면서 다음 상태로 이동하고,
리턴하면서 진행 상황을 차례차례 복구한다.

tmp = [0, a, b, c, d]
현재 남아있는 재료의 수

2 개의 사용 위치

```
def backTrack(t):  
    global res  
    if tmp[1] == tmp[2] == tmp[3] == tmp[4] == 0:  
        res += 1  
        return  
    for i in range(1, 5):  
        if i != t and tmp[i] > 0:  
            tmp[i] -= 1  
            backTrack(i)  
            tmp[i] += 1
```



03

구현하기

예제: 부분수열의 합 (#1182)

이 문제는 가지치기를 할 요소가 없기 때문에
백트래킹 대신 2^N 브루트포스를 해도 방문 횟수는 같다.
대신 백트래킹으로 연산 횟수를 줄일 수 있다.

먼저 비트마스킹으로 브루트포스를 해보자.
원소의 개수가 N 일 때, 각 원소의 상태는
부분수열에 포함되거나/포함되지 않거나 둘 중 하나이다.



03

구현하기

이런 상태는 $0 \sim 2^N - 1$ 루프를 돌리면 표현이 가능하다.
어째서일까?

메모리 내의 모든 자료는 이진수로 저장되어 있다.

$N = 8$ 일 때

$240 = (11110000)_2$, $14 = (00001110)_2$

각 자리가 1이면 부분수열에 포함되고,

0이면 포함되지 않는 상태라고 정의하면

$0 \sim 2^N - 1$ 범위 내의 이진수로 모든 상태를 표현할 수 있다.


```
def solve():
    ret = 0
    for i in range(1, 2**N):
        s = 0
        for j in range(N):
            if i & (1 << j):
                s += arr[j]
        if s == S:
            ret += 1
    return ret
```

$n \ll k$: n (이진수)를 왼쪽으로 k 칸 이동
즉 $1 \ll k == 2^k$, k 번째 자리만 1인 상태
여기서 i 와 $\text{and}(\&)$ 연산자를 사용하면
 i 의 k 번째 자리가 1이면 True,
0이면 연산 결과 False가 된다.
즉 i 의 k 번째 비트가 무엇인지 알 수 있다.



03

구현하기

$$\begin{array}{r} 14 = (00001110)_2 \\ \& \quad 1 = (00000001)_2 \\ \hline 0 = (00000000)_2 \end{array}$$

$$14 = (00001110)_2$$

$$\& (1 \ll 1)_2 = (00000010)_2$$

$$2 = (00000010)_2$$



03

구현하기



이번에는 백트래킹으로 구현해보자!

`backTrack(i, s)` : $i-1$ 번째 자리까지 확인했을 때 합이 s 인 상태

$i-1$ 번째 자리까지 확인한 상태이므로
다음 상태에서는 $i \sim N-1$ 번째 원소 중 하나를 추가해서 호출한다.

VS

i 번째 자리를 확인할 때
 i 번째를 포함하는 경우, 포함하지 않는 경우 둘 중 하나로 나눠서 호출한다.

```
import sys
input = sys.stdin.readline

N, S = map(int, input().split())
arr = list(map(int, input().split()))
res = 0
def backTrack(i, s):
    global res
    if i > 0 and s == S:
        res += 1
    for j in range(i, N):
        backTrack(j+1, s+arr[j])

backTrack(0, 0)
print(res)
```

$i-1$ 번째 자리까지 확인한 상태이므로
다음 상태에서는 $i \sim N-1$ 번째 원소 중 하나를 추가해서 호출한다.

VS

i 번째 자리를 확인할 때
 i 번째를 포함하는 경우, 포함하지 않는 경우 둘 중 하나로 나눠서 호출한다.
이것은 숙제로 남겨 두겠습니다...

결론적으로 두 코드의 실행 시간을 비교해보면,
백트래킹이 더 빠르다.
왜냐하면 백트래킹은 트리 형태로 연산을 반복하기 때문에
이전에 했던 연산을 또 하지 않아서 계산상으로 효율적이다.

하지만 함수 호출이라는 작업 자체가 다른 작업보다 메모리나 시간을
상당히 많이 소비한다.
그래서 더 좋은 방법이 있는데...

4주차 다이나믹 프로그래밍에서 계속...



강의 설문조사

QR 제공 예정

Thank you For Listening

3주차 스터디를 들어주셔서 감사합니다.
뒷풀이에 참여하실 분들만 강의실에 남아주세요!

—

